

Task Manager - Deployment Configuration Migration Report

An analysis and walkthrough of structural fixes to the Docker Compose and Gateway environments.

1. Executive Summary

To run the Task Manager Multi-Tenant ecosystem successfully inside Docker containers, several corrections were required. The original configuration contained invalid project paths, missing builds, and communication mismatches. We restructured the deployment code by creating project-specific container configurations, enabling environment-specific routing at the gateway level, and defining healthy dependency sequences.

2. File Action & Status Summary

File Name / Path	Action	Purpose
TaskManagerService/Dockerfile	CREATED	Standardized container recipe for the Task Manager API service.
AuthenticationService/Dockerfile	CREATED	Standardized container recipe for the Identity/Auth service.
APIGateway/Dockerfile	CREATED	Standardized container recipe for the Ocelot API Gateway.
APIGateway/Ocelot.Production.json	CREATED	Ocelot configurations resolving internal Docker networks and ports.
.env (Root)	CREATED	Centralized default environment secrets and database passwords.
APIGateway/Program.cs	MODIFIED	Added environment-aware JSON loading logic (e.g. Ocelot.Production.json).
APIGateway/APIGateway.csproj	MODIFIED	Configured MSBuild to copy Ocelot configs to output directory.
docker-compose.yml	MODIFIED	Wired all services, added database healthcheck, mapped env parameters.
Dockerfile (Root)	DELETED	Removed the outdated root file pointing to a non-existent project.

3. Detailed File Breakdown & Real-Time Analogies

A. Project-Specific Dockerfiles

Why we created them: In microservice environments, compiling each project on developers' individual computers often causes issues due to mismatches in installed SDK versions, local operating systems, or environmental patches. We created separate Dockerfiles for *TaskManagerService*, *AuthenticationService*, and *APIGateway*. This isolates the runtime environment, guaranteeing it behaves exactly the same on your PC, staging servers, and cloud environments.

Real-Time Analogy: The Meal Kit (e.g., HelloFresh)

Instead of a chef sending raw ingredients and hoping the home cook has the exact same cooking pans, specific spices, and stove temperatures, a meal kit comes with portioned items and a strict, repeatable instruction sheet. No matter who cooks it or what kitchen they use, the final dish tastes identical every time because the ingredients and steps are encapsulated.

B. Ocelot.Production.json & Environment-Aware Routing

Why we created them: The original gateway was set up to direct traffic downstream to localhost. While this works locally during debugging, it fails inside Docker because localhost inside a container points to that container itself, not the other services. We added Ocelot.Production.json to route calls via the Docker Bridge network using service hostnames (like taskmanager-api and taskmanager-auth) over HTTP.

Real-Time Analogy: The Office Receptionist & Intercom

If a customer calls an office receptionist (API Gateway) from outside and asks for the Sales Department, the receptionist doesn't dial the company's external telephone number again. Instead, they use the internal intercom line and transfer you to Extension 102 (internal service hostname). If they tried to dial the external line (localhost), the system would block the transfer.

C. Docker Compose Update & Database Healthcheck

Why we updated them: A multi-container system has complex boot sequences. If the APIs try to start and check credentials before the SQL Server container has finished booting, they will throw database exceptions and crash. We added SQL Server healthchecks to compose, forcing the API services to wait until the database is fully operational. We also mapped the environment variables DB_LOCAL and DB_PROD properly to point to the SQL Server container.

Real-Time Analogy: The Symphony Conductor

If the violinists start playing before the stage is set, the instruments are tuned, or the conductor is present, the performance fails. Docker Compose is the symphony conductor. By configuring healthchecks, the conductor ensures the database is fully 'tuned' and ready before signaling the APIs to start playing.

D. Root .env File

Why we created it: Hardcoding passwords and JWT secrets inside docker-compose files is a severe security risk. By moving these configuration parameters into a .env file, we separate configuration from execution code. Developers can modify passwords locally without altering the compose file itself.

Real-Time Analogy: The Safety Deposit Box

Instead of writing passwords and security card keys on sticky notes and gluing them to the office door, you put them all in a centralized, secure deposit box. Whenever a staff member (a container service) needs access, they look into the deposit box rather than having the credentials baked directly into their employee handbook.